Budapesti Műszaki Szakképzési Centrum

Bolyai János Műszaki Technikum és Kollégium

NapiCsepp

DailyDrop - Szokáskövető és produktivitáskezelő

Vizsgaremek dokumentáció

Készítette:

Evbuomwan Abigél Osaretin · Sipos Fruzsina

2025/2026. tanév

Tartalom

Bevezetés 4

A projekt neve – NapiCsepp 4

A projekt célja 4

Probléma leírása 5

Célcsoport 6

Fejlesztői dokumentáció 6

Fejlesztési folyamat és feladatmegosztás 6

Fejlesztés során használt technológiák 7

Backend technológiák 7

Frontend technológiák 8

Adatbázis és adatkezelés 8

Konténerizáció és futtatási környezet 8

Verziókövetés és fejlesztési folyamat 9

Autentikáció és biztonság (JWT) 9

Összegzés 9

Adatbázis dokumentáció 9

Táblák 9

Kapcsolatok 10

Táblákhoz tartozó oszlopok 10

Backend architektúra összefoglaló 12

Rétegek szerepe 12

Modulok áttekintése 13

REST API dokumentáció 15

Bevezetés 15

API működési áttekintés 15

Autentikáció és JWT – Login folyamat 17

Az Activities API szerepe és működése 18
Activities végpontok 18
Frontend komponensek bemutatása 24
Sidebar komponens 25
AddTaskPopup komponens 26
EventPopup komponens 27
Toast és showToast segédkomponens 28
Manuális tesztek 29
Backend manuális tesztek 29
Frontend manuális tesztek 32
Automatizált tesztek 34
Automatizált backend tesztek 34
Automatizált frontend tesztek 39
Felhasználói dokumentáció 41
Szerepkörök bemutatása 41
Felhasználó (User) 41
Moderátor (Moderator) 41
Adminisztrátor (Admin) 41
Esetek bemutatása 42
Regisztráció 42
Bejelentkezés 43
Napi feladatok kezelése 43
Szokások követése 44
Naptár és eseménykezelés 45
Statisztikák 46
Profil kezelése 46
Admin panel részletes kezelési útmutató 46

Összefoglalás 48
Kitekintés 49
Felhasznált források és szoftver bibliográfia 49
Mellékletek 51

Bevezetés

A projekt neve – NapiCsepp

A NapiCsepp egy modern, webalapú szokáskövető és produktivitáskezelő alkalmazás, amelyet arra terveztek, hogy segítsen a felhasználóknak napi rutinjuk felépítésében és fenntartásában.

A projekt neve két szóból tevődik össze. A 'Napi' előtag az alkalmazás mindennapi, rendszeres használatára utal arra, hogy a felhasználó naponta nyisson rá, naponta jelölje be a teljesített szokásait, és naponta lássa, merre tart.

A 'Csepp' szó egy mélyebb gondolatot hordoz: ahogyan egy apró vízcsepp önmagában jelentéktelennek tűnhet, de sok csepp együtt patakot, folyót, tengert alkot, úgy működik az emberi fejlődés is. Egy nap nem változtat sokat, de ha minden nap teszünk egy kis lépést, az hetek, hónapok, évek alatt óriási különbséget jelent. A NapiCsepp nevének üzenete tehát: ne a nagy áttörésekre várj, hanem bízz a kis, de rendszeres cseppekben azok visznek messzire.

A projekt célja

A NapiCsepp projekt célja egy modern, felhasználóbarát, reszponzív webalkalmazás létrehozása, amely egységes felületen teszi lehetővé a napi feladatok kezelését, a hosszú távú szokások követését, az eseménynaptár használatát és a haladás statisztikai elemzését.

A projekt főbb célkitűzései:

- Egységes platform a négy főmodul (feladatok, szokások, naptár, statisztikák) számára, hogy a felhasználónak egyetlen alkalmazást kelljen megnyitni a napi produktivitás menedzseléséhez.
- Szokáskövetés meghatározott időtartammal: a rendszer automatikusan számlálja az elvégzett napokat, és jelzi a haladást.
- Napi feladatkezelés nehézség és típus besorolással a valódi teljesítményképhez.
- Részletes statisztikák napi és heti szinten, nehézségi eloszlással és diagrammal.
- Szerepköralapú hozzáférés-kezelés (User, Moderátor, Admin).
- Reszponzív megjelenés mobil és asztali nézetben egyaránt.
- Docker Compose alapú egyparancs-telepítés.

A NapiCsepp elnevezés nem csupán kitalált: a mögötte meghúzódó szemlélet az alkalmazás szívében is jelen van. Minden egyes nap, amelyen a kliens megpróbálja teljesíteni a céljait, egy-egy csepp. Az alkalmazás erre a hosszú távú, növekvő gondolkodásmódra épít.

Probléma leírása

A modern életritmus egyik legjelentősebb kihívása a szokások kialakítása és fenntartása. Kutatások szerint egy új szokás rögzüléséhez átlagosan 66 napra van szükség, mégis a legtöbb ember heteken belül felhagy a kitűzött céljaival. Ennek egyik fő oka az, hogy hiányoznak az egyszerűen használható, motiváló eszközök, amelyek segítenek nyomon követni a mindennapi célokat.

A meglévő megoldások egyszerű teendőlisták vagy általános naptáralkalmazások nem képesek összekötni a napi feladatkezelést a hosszú távú szokáskövetéssel. Ezek az eszközök jellemzően nem mutatják meg vizuálisan a haladást, nem különböztetik meg az egyszeri feladatokat a visszatérő szokásoktól, és nem adnak statisztikai visszajelzést a felhasználó teljesítményéről.

Napjainkban számos digitális eszköz ígéri, hogy segít a szokások kialakításában, azonban ezek jelentős része vagy túlzásba esik (komplex játékos szemléletű rendszerek), vagy túl egyszerű marad (sima jegyzetfüzet visszajelzés nélkül). A NapiCsepp ezt a kettős problémát oldja meg: középutas módon nyújt áttekinthető felületet, kellő vizualizációt és valódi motiváló hatást.

További probléma, hogy a meglévő megoldások jellemzően szigetelten működnek: van egy külön naptár, egy külön todo-alkalmazás, egy külön szokáskövető. A NapiCsepp ezt az összetettséget oldja meg azzal, hogy mindent egyetlen felületen elérhetővé tesz.

Célcsoport

Az alkalmazás elsődleges célcsoportja azok az emberek, akik több struktúrát keresnek a mindennapjaikban:

- Diákok és egyetemisták, akik tanulási szokásokat, edzésrendet vagy más napi rutint szeretnének kialakítani és fenntartani.
- Dolgozók, akik a munka-magánélet egyensúlyát szokáskövetéssel és naptárkezeléssel kívánják javítani.
- Sport iránt érdeklődők, akik edzés- és regenerációs szokásaikat követik nyomon.
- Bárki, aki célokat tűz ki magának és vizuális visszajelzést szeretne a haladásáról legyen az olvasás, meditáció, vízivás vagy bármilyen más napi tevékenység.

Fejlesztői dokumentáció

Fejlesztési folyamat és feladatmegosztás

A projekt megvalósítása során a fejlesztési feladatok két fő terület mentén kerültek felosztásra, figyelembe véve a csapattagok erősségeit és érdeklődési körét. A munkaszervezés célja az volt, hogy a frontend és backend fejlesztés jól elkülöníthető módon, ugyanakkor szoros együttműködésben valósuljon meg.

A frontend fejlesztésért Evbuomwan Abigél volt felelős, aki a felhasználói felület kialakítását, a React alapú komponensek implementálását, valamint a vizuális megjelenés megtervezését végezte. Emellett a frontendhez kapcsolódó tesztelési feladatokat is ellátta, valamint részt vett a felhasználói dokumentáció elkészítésében is.

A backend fejlesztési feladatokat Sipos Fruzsina látta el, aki a szerveroldali logika kialakításáért, az API végpontok implementálásáért és az adatbázissal való

kommunikáció megvalósításáért felelt. Feladatai közé tartozott továbbá az adatbázis struktúrájának megtervezése, valamint a backend működésének tesztelése és a fejlesztői dokumentáció elkészítése is.

A két terület folyamatos egyeztetés mellett haladt. Különös figyelmet fordítottak arra, hogy a frontend és backend közötti kommunikáció egységes és jól definiált legyen. Az API végpontok és az adatstruktúrák kialakítása közös döntések alapján történt, így biztosítva a rendszer elemeinek megfelelő illeszkedését.

A projekt során alkalmazott verziókezelés lehetővé tette a párhuzamos munkavégzést, valamint a módosítások nyomon követését. A feladatmegosztás ellenére a csapattagok szükség esetén egymás munkáját is segítették, így biztosítva a fejlesztés hatékony és folyamatos előrehaladását.

Fejlesztés során használt technológiák

Backend technológiák

A szerveroldali logika megvalósításához a Node.js futtatókörnyezet került alkalmazásra, amely esemény vezérelt és nem blokkoló működésének köszönhetően hatékonyan képes kezelni a párhuzamos kéréseket. Ez különösen fontos webalkalmazások esetén, ahol több felhasználó egyidejű kiszolgálása történik. A Node.js használata mellett szólt az is, hogy JavaScript-alapú környezetet biztosít, így a frontend és backend fejlesztése egységes technológiai alapon valósulhat meg.

A projekt TypeScript nyelven készült, amely a JavaScript kiterjesztéseként statikus típuskezelést biztosít. Ennek köszönhetően a fejlesztés során számos hiba már fordítási időben kiszűrhető, ami növeli a rendszer stabilitását. A típusok használata emellett átláthatóbbá teszi a kódot, és megkönnyíti a későbbi módosításokat, különösen nagyobb méretű alkalmazások esetén.

A backend felépítése az Express.js keretrendszerre épül, amely lehetővé teszi egy jól strukturált REST API kialakítását. Az alkalmazás különböző végpontokon keresztül biztosítja az adatok elérését, ahol minden kérés egy meghatározott feldolgozási láncon halad végig. Ebben a folyamatban fontos szerepet játszanak a middleware-ek, amelyek például az autentikáció ellenőrzését vagy a bemeneti adatok validálását végzik. Ez a megközelítés modularizált és könnyen bővíthető architektúrát eredményez.

Frontend technológiák

A felhasználói felület React könyvtár segítségével készült, amely egy modern, komponens-alapú megközelítést alkalmaz. Ennek során a felület kisebb, egymástól független egységekből épül fel, amelyek külön-külön is kezelhetők és újra felhasználhatok. Ez a struktúra nemcsak átláthatóbbá teszi a kódot, hanem lehetővé teszi a gyorsabb fejlesztést és hibakeresést is.

A React működésének egyik fontos eleme az állapotkezelés, amely meghatározza, hogy a felhasználói interakciók hatására hogyan változik a megjelenített tartalom. Az alkalmazás a backenddel API-hívásokon keresztül kommunikál, így az adatok dinamikusan kerülnek betöltésre és frissítésre. Ez biztosítja, hogy a felhasználó mindig az aktuális állapotot lássa anélkül, hogy az oldalt újra kellene tölteni.

A fejlesztési folyamatot a Vite build eszköz támogatja, amely jelentősen felgyorsítja a munkát. A gyors újra fordítás és az azonnali visszajelzés lehetővé teszi a fejlesztő számára, hogy rövid idő alatt kipróbálja a módosításokat. Ez nemcsak kényelmi szempontból előnyös, hanem növeli a fejlesztési hatékonyságot is.

Adatbázis és adatkezelés

Az alkalmazás adatainak tárolása relációs adatbázisban történik, amelyhez MySQL került kiválasztásra. A relációs modell lehetővé teszi az adatok strukturált kezelését, valamint a különböző entitások közötti kapcsolatok egyértelmű definiálását. Ez különösen fontos olyan rendszerekben, ahol az adatok egymásra épülnek.

Az adatbázis és a backend közötti kommunikáció során SQL lekérdezések kerülnek alkalmazásra, amelyek segítségével az adatok lekérdezése, módosítása és törlése történik. A backend feladata, hogy biztosítsa az adatok konzisztenciáját, valamint megakadályozza a hibás vagy jogosulatlan módosításokat. Az adatkezelés során fontos szempont az is, hogy a lekérdezések hatékonyak legyenek, így az alkalmazás nagyobb adatmennyiség esetén is megfelelő teljesítményt nyújtson.

Konténerizáció és futtatási környezet

Az alkalmazás konténerizált formában fut Docker segítségével, amely egységes futtatási környezetet biztosít. Ez azt jelenti, hogy a fejlesztői, tesztelési és éles környezet között nem alakulnak ki eltérések, így elkerülhetők az úgynevezett „nálam működik” típusú problémák. A konténerizáció lehetővé teszi az alkalmazás gyors telepítését és egyszerűbb skálázását is.

A Docker használata hozzájárul a rendszer modularitásához, mivel az egyes szolgáltatások külön konténerekben futtathatók. Ez a megközelítés megkönnyíti a rendszer karbantartását és továbbfejlesztését.

Verziókövetés és fejlesztési folyamat

A fejlesztés során Git alapú verziókövetés került alkalmazásra, a projekt tárolása GitHub platformon történik. Ez lehetővé teszi a kódváltozások részletes nyomon követését, valamint biztosítja, hogy a fejlesztési folyamat visszakövethető legyen. A verziókezelés támogatja a párhuzamos fejlesztést is, így több funkció egyidejűleg dolgozható ki anélkül, hogy azok egymást zavarnák.

A verziókövetés szerepe nemcsak a fejlesztés során fontos, hanem hibakeresés esetén is, mivel lehetővé teszi korábbi állapotok visszaállítását.

Autentikáció és biztonság (JWT)

A rendszer biztonságos működésének érdekében a felhasználók azonosítása JSON Web Token alapú autentikációval történik. Ez egy állapotmentes megoldás, amely során a szerver nem tárolja a felhasználói session adatokat, hanem minden szükséges információt a token tartalmaz. A token létrehozása a bejelentkezés során történik, amikor a felhasználó megadja az azonosításhoz szükséges adatokat.

A token tartalmazza a felhasználó azonosítóját, valamint az érvényességi időre vonatkozó adatokat, így biztosítható, hogy a hozzáférés időben korlátozott legyen. A frontend a tokent eltárolja, és minden védett kérés során elküldi a szerver felé. A backend minden egyes ilyen kérésnél ellenőrzi a token érvényességét, és csak sikeres ellenőrzés esetén engedélyezi a hozzáférést.

Ez a megoldás növeli a rendszer biztonságát, mivel megakadályozza a jogosulatlan hozzáférést, és biztosítja, hogy csak hitelesített felhasználók végezhessenek műveleteket.

Összegzés

A projekt során alkalmazott technológiai stack egy modern, jól strukturált és hatékony rendszert eredményez. A backend és frontend közötti világos feladatmegosztás, valamint az adatkezelés és autentikáció megfelelő kialakítása biztosítja az alkalmazás megbízható működését. A választott technológiák nemcsak a jelenlegi követelményeknek felelnek meg, hanem megfelelő alapot nyújtanak a rendszer későbbi bővítéséhez is.

Adatbázis dokumentáció

Táblák

Users: Ez a tábla lesz felelős tárolni a felhasználó adatait

Activities: Ez a tábla fogja tárolni a napi teendőlista feladatait és a szokásokat

Events: Ez a tábla fogja kezelni a felhasználó által megadott eseményeket

Types: Ez fogja a taskok és habitokhoz tartozó típusokat tartalmazni. (mozgás, házimunka, tanulás)

Difficulties: Ez fogja a taskok és habitokhoz tartozó nehézségi szinteket tartalmazni. (könnyű, közepes, nehéz)

Kapcsolatok

Users és az Activities között: 1: N Egy felhasználó több teendőt vagy szokást felvehet és egy teendőhöz vagy szokáshoz egy felhasználó tartozhat

Users és a Events között: 1: N Egy felhasználó több eseményt is felvehet és egy eseményhez egy felhasználó tartozik

Activity és a Types között: N:1 A teendőkhöz csak egy típus tartozhat.

Activity és a Difficulties között: N:1 A teendőkhöz csak egy nehézség tartozhat.

Táblákhoz tartozó oszlopok

Users

user_id: INT Elsődleges kulcs a felhasználó egyedi azonosítóját tartalmazza

username: VARCHAR A felhasználó nevét tartalmazza

password: VARCHAR A felhasználó jelszavát tartalmazza titkosítva

email: VARCHAR A felhasználó email címét tartalmazza

language: VARCHAR A felhasználó által kiválasztót nyelv (jelenleg csak magyar)

role: VARCHAR A felhasználó jogosultságának a neve (user, admin, moderator)

register_date: DATETIME A felhasználó regisztrálásának dátuma

Activities

activity_id: INT Elsődleges kulcs a teendő egyedi azonosítóját tartalmazza

user_id: INT Idegen kulcs a felhasználó azonosítója

activity_name: VARCHAR A teendő nevét tartalmazza

activity_type_id: INT Idegen kulcs a típus egyedi azonosítóját tartalmazza

activity_difficulty_id: INT Idegen kulcs a nehézségi szint egyedi azonosítóját tartalmazza

activity_achive: BOOLEAN, Ha elkészítette a teendőt akkor igaz értéke lesz

activity_start_date: DATE Azt a napot fogja tartalmazni amelyikhez beírta, hogy kezdődjön

activity_end_date: DATE Azt a napot fogja tartalmazni amelyikhez beírta, hogy végződjön

progress_counter: INT Azt tartalmazza, hogy hány napon sikerült teljesíteni a szokást

Types

type_id: INT Elsődleges kulcs a típus egyedi azonosítóját tartalmazza

type_name: VARCHAR A típus nevét tartalmazza

Difficulties

difficulty_id: INT Elsődleges kulcs a nehézségi szint egyedi azonosítóját tartalmazza

difficulty_name: VARCHAR A nehézségi szint nevét tartalmazza

Events

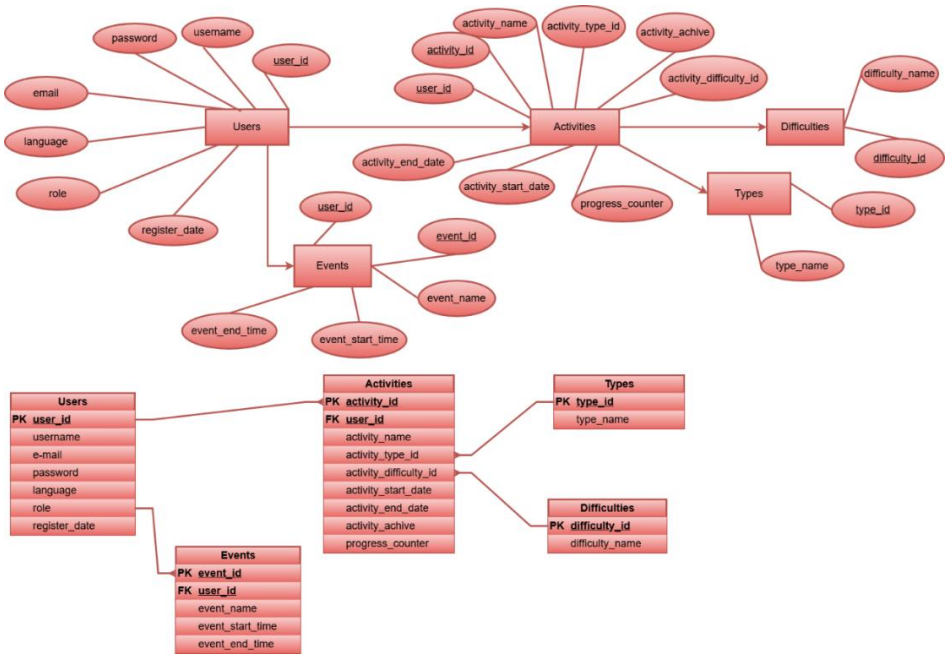
event_id: INT Elsődleges kulcs az esemény egyedi azonosítóját tartalmazza

user_id: INT Idegen kulcs az eseményt létrehozó felhasználó azonosítója

event_name: VARCHAR Az esemény nevét tartalmazza

event_start_time: DATETIME Az esemény kezdő időpontját tartalmazza

event_end_time: DATETIME Az esemény végének időpontját tartalmazza



1. Adatbázis diagramok ER és relációs

Backend architektúra összefoglaló

Az alkalmazás backendje klasszikus, rétegzett architektúrát követ, amely három fő rétegre épül: Repository, Service és Controller. Ez a felépítés biztosítja az átláthatóságot, a karbantarthatóságot és a skálázhatóságot.

Rétegek szerepe

A Repository réteg felelős az adatbázissal való közvetlen kommunikációért. Itt történik az adatok lekérése, mentése, frissítése és törlése (CRUD műveletek). Ez a réteg elszigeteli az adatkezelést az üzleti logikától.

A Service réteg tartalmazza az üzleti logikát. Itt történik az adatok validálása, a jogosultságok ellenőrzése, valamint az üzleti szabályok érvényesítése. A service réteg a repository-t használja adatműveletekhez, de nem kommunikál közvetlenül a klienssel.

A Controller réteg kezeli az API végpontokat és a kliens kéréseit. Fogadja a HTTP kéréseket, meghívja a megfelelő service metódusokat, majd visszaküldi a válaszokat megfelelő státuszkódokkal.

Modulok áttekintése

User Modul

A User modul a felhasználók kezeléséért felel, beleértve a hitelesítést és a jogosultságkezelést. A Repository réteg biztosítja a felhasználókhoz kapcsolódó adatbázis-műveleteket, mint például a bejelentkezés ellenőrzése, a CRUD műveletek végrehajtása, az email címek ellenőrzése, valamint a moderátorok lekérése. A Service réteg végzi az üzleti logika kezelését, ideértve az adatok validálását, a JWT tokenek generálását, valamint a felhasználói jogosultságok ellenőrzését. A Controller réteg kezeli az API végpontokat, például a bejelentkezést, regisztrációt, profil lekérését, felhasználók listázását, valamint a módosítási és törlési műveleteket. A Model egységes adatstruktúrát biztosít, amely tartalmazza többek között a felhasználó azonosítóját (user_id), nevét, email címét és szerepkörét.

Event modul

Az Event modul az időhöz kötött események kezeléséért felel az alkalmazásban. A Repository réteg biztosítja az eseményekkel kapcsolatos adatbázis-műveleteket, beleértve az események lekérését, új események létrehozását tranzakciók segítségével, valamint azok módosítását és törlését. A Service réteg feladata az adatok validálása és a hibák kezelése, biztosítva, hogy csak érvényes és konzisztens események kerüljenek feldolgozásra. A Controller réteg az eseményekhez tartozó API végpontokat kezeli, és lehetővé teszi a CRUD (create, read, update, delete) műveletek végrehajtását a kliens számára. Az Event modell az események alapvető adatait tartalmazza, mint például az esemény neve, kezdési időpontja és befejezési ideje.

Activity modul

Az Activity modul a felhasználók feladatainak és szokásainak kezelésére szolgál. A Repository réteg kezeli az aktivitások adatbázisból történő lekérését, valamint a típusok és nehézségi szintek kezelését, továbbá dinamikus frissítési műveleteket biztosít az adatok módosításához. A Service réteg felel az adatok validálásáért, az aktivitások

létrehozásáért, módosításáért és törléséért, valamint a hibák megfelelő kezeléséért. A Controller réteg biztosítja az aktivitásokhoz és szokásokhoz tartozó API végpontokat, és lehetővé teszi ezek kezelését a kliens számára. Az Activity modell tartalmazza az aktivitás nevét, típusát, nehézségét, állapotát (teljesítve-e), az időintervallumot, valamint az előrehaladást jelző értéket.

Difficulty és Type modulok

A Difficulty és Type modulok segédadatok kezelésére szolgálnak, mint például a nehézségi szintek és a különböző kategóriák. Ezek a modulok egyszerű felépítésűek: a Repository réteg az adatok lekéréséért felel, míg a Controller rétegen keresztül ezek az adatok API-n keresztül elérhetővé válnak a kliens számára. A Model ezekben az esetekben egyszerű struktúrát követ, amely az adott elem azonosítóját és nevét tartalmazza.

Overview modul

Az Overview modul a felhasználó napi szintű tevékenységeinek összesítését biztosítja. A Repository réteg egy tárolt eljárás segítségével gyűjti össze az adatokat az adatbázisból. A Service réteg feladata az adatok feldolgozása és konzisztens formába rendezése. A Controller réteg API végponton keresztül továbbítja ezeket az adatokat a kliens felé. Az Overview modell tartalmazza a dátumot, valamint az adott naphoz tartozó aktivitások, szokások és események számát.

Statistic modul

A Statistic modul a rendszer- és felhasználószintű statisztikai adatok kezeléséért felel. A Repository réteg tárolt eljárások segítségével gyűjti össze a szükséges statisztikai adatokat az adatbázisból. A Service réteg ezeket az adatokat feldolgozás nélkül vagy minimális logikával továbbítja. A Controller réteg API végpontokon keresztül biztosítja a statisztikák elérését, és gondoskodik a megfelelő válaszformátumról. A

Modellek tartalmazzák mind a rendszer szintű (például összes felhasználó vagy aktivitás), mind a felhasználóra vonatkozó statisztikai adatokat.

Összegzés

A backend rendszer jól strukturált, rétegzett architektúrát követ, amely világosan elkülöníti: az adatkezelést (Repository), az üzleti logikát (Service), és a kliens kommunikációt (Controller).

Az egyes modulok (User, Event, Activity, stb.) egységes mintát követnek, ami megkönnyíti a fejlesztést és a bővítést. A rendszer támogatja a jogosultságkezelést, a validációt, a tranzakciókat és a skálázható működést, így stabil alapot biztosít egy komplex alkalmazás számára.

REST API dokumentáció

Bevezetés

A NapiCsepp alkalmazás backendje REST API-kra épül, amelyek a frontend és a szerver közötti kommunikációt biztosítják. Az API feladata, hogy a felhasználói műveleteket és az adatok kezelését strukturált módon tegye lehetővé, miközben biztosítja a biztonságot és a rendszer konzisztenciáját.

Az API főbb funkciói közé tartozik a felhasználói autentikáció, az aktivitások létrehozása, lekérdezése, módosítása és törlése, valamint a háttérfolyamatokhoz kapcsolódó adatok kezelése.

A dokumentáció célja, hogy bemutassa az API kulcsfontosságú végpontjait, azok működését, a frontend-backend kommunikáció menetét, valamint a hibakezelést és a különböző szerepkörök szerint várható viselkedést. A későbbi fejezetekben a főbb endpointok részletes leírása és példák segítségével szemléltetjük az alkalmazás működését a gyakorlati felhasználás kontextusában.

API működési áttekintés

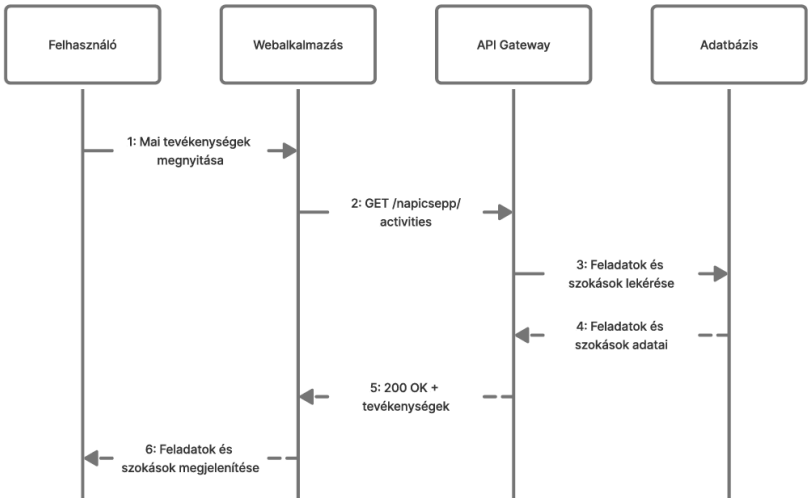
A NapiCsepp alkalmazás a REST (Representational State Transfer) elv alapján az alkalmazás végpontok (endpointok) segítségével fogadja a kliens kéréseit, dolgozza

fel azokat, majd visszaküldi a megfelelő válaszokat JSON formátumban. A felhasználó műveletei például aktivitás létrehozása vagy lekérdezése a frontend komponensein keresztül érkeznek a szerverhez HTTP kérések formájában. A backend a beérkező kéréseket az adott endpointokhoz irányítja, ahol a logika feldolgozza az adatokat, végrehajtja a szükséges műveleteket az adatbázisban, majd visszaküldi a választ a kliensnek.

A kommunikáció alapját a JSON formátumú üzenetek jelentik, amelyek a kérésben (request) és a válaszban (response) is használatosak. Minden egyes védett végpont esetén a felhasználó azonosítására a JSON Web Token (JWT) alapú autentikáció szolgál, amely biztosítja, hogy csak hitelesített felhasználók férhessenek hozzá az adatokhoz.

A rendszerben az adatok áramlása a következőképpen valósul meg: a felhasználó a felületen keresztül kezdeményezi a műveletet, a frontend elküldi a kérést a backend felé, ahol a middleware-ek ellenőrzik a kérés jogosultságát, majd az üzleti logika feldolgozza a kérést. A backend ezután az adatbázisban végrehajtja a szükséges műveletet, és a feldolgozás eredményét JSON formátumban visszaküldi a frontendnek, amely frissíti a felhasználói felületet.

A dokumentációban szereplő sequence diagram a fő adatáramlási folyamatot szemlélteti. A diagramon jól látható, hogy a frontend és backend közötti kommunikáció több lépésből áll.



2. ábra Sequence diagram részlet

Autentikáció és JWT – Login folyamat

A NapiCsepp alkalmazásban a felhasználói azonosítás JSON Web Token (JWT) alapú autentikációval történik. Ez az állapotmentes (stateless) megoldás biztosítja, hogy a bejelentkezett felhasználók hitelesítetten férhessenek hozzá a védett API végpontokhoz anélkül, hogy a szerver oldalon session-t kellene tárolni.

Login folyamat

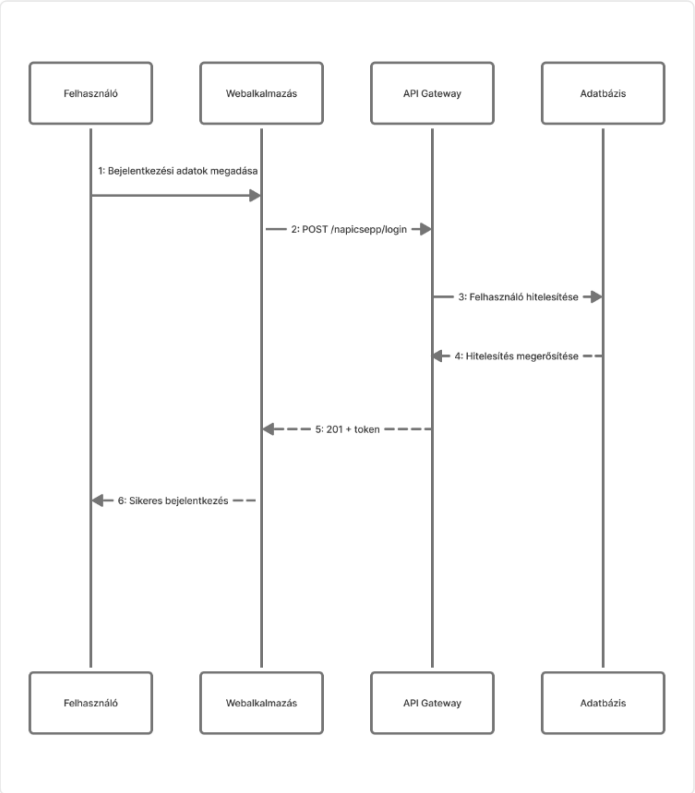
A felhasználó a felhasználói felületen keresztül megadja az email címét és jelszavát, majd a frontend POST kérést küld a /login végpontra. A backend ellenőrzi a megadott adatokat az adatbázisban, és sikeres azonosítás esetén előállítja a JWT tokent. A token tartalmazza a felhasználó egyedi azonosítóját (user_id), a kiállítás idejét (iat) és a lejárati időt (exp), amelyet a rendszer alapértelmezés szerint 2 órára állít be.

A token visszaküldése után a frontend a kapott értéket a böngésző localStorage-ában tárolja, és minden védett API híváshoz a kérés fejlécébe (x-access-token) helyezi. A backend middleware minden védett kérésnél ellenőrzi a token érvényességét, kibontja a payload-ot, és elérhetővé teszi a req.user objektumban a controller számára.

Hibakezelés

A JWT alapú autentikáció során előforduló leggyakoribb hibaüzenetek:

- 401 Érvénytelen token: a token aláírása nem egyezik, vagy a token lejárt, ilyenkor a szerver visszaküldi az üzenetet: „Az auth nem sikerült!”. 403 Token hiányzik: a kérés nem tartalmazza a x-access-token fejlécet, a szerver ilyenkor a „Token szükséges” választ adja.



3. ábra Sequence diagramm login

Az Activities API szerepe és működése

Az Activities entitás a NapiCsepp alkalmazás fő funkciója, amely lehetővé teszi a napi tevékenységek és szokások kezelését. Az API ezen a funkción keresztül valósítja meg az összes alapvető műveletet: lekérdezést, létrehozást, módosítást és törlést (GET, POST, PUT, DELETE). Minden többi végpont erre az alapra épül.

A felhasználók a belépés után kapnak egy biztonsági tokent (JWT), melyet minden kéréshez továbbítanak. A szerver ezt a tokent ellenőrzi, és csak érvényes hitelesítés esetén engedélyezi a műveleteket.

Activities végpontok

Activities lekérdezése

Az /napicsepp/activities végpont a NapiCsepp alkalmazás egyik alapvető funkciója, amely lehetővé teszi a felhasználók számára, hogy áttekintsék az összes rögzített napi aktivitásukat. Ez az endpoint a teljes alkalmazás működésének kiindulópontja,

hiszen a felhasználói tevékenységek kezelése köré épül a többi funkció, például a szokások követése vagy az egyes aktivitások részleteinek módosítása.

A folyamat a következőképpen zajlik: a felhasználó a belépést követően megkapja a JWT alapú hitelesítő tokent, amelyet minden API kéréshez továbbít. Amikor a kliens lekéri az aktivitáslistát, a szerver első lépésként ellenőrzi a tokent, hogy az érvényes-e és a felhasználó jogosult-e a kérés végrehajtására. Ezzel biztosítja, hogy csak az adott felhasználó adataihoz férhessen hozzá más ne tudja lekérni a listát.

A szerver a token érvényességének ellenőrzése után az adatbázisból lekérdezi a felhasználó aktivitásait. Minden aktivitás részletesen tartalmazza a nevét, típusát, nehézségi szintjét, a tevékenység kezdő- és záródátumát, valamint a haladás mértékét. Ezen információk segítségével a felhasználó könnyen áttekintheti, hogy mely tevékenységek vannak folyamatban, melyeket kell frissíteni, illetve melyeket már befejezett.

Az endpoint különböző válaszokkal reagálhat, attól függően, hogy a kérés sikeres volt-e vagy sem:

200 OK: A lekérdezés sikeres volt, és a felhasználó összes aktivitását visszakapja a kliens.

401 Unauthorized: A token érvénytelen, lejárt vagy hiányzik, ezért a rendszer megtagadja a hozzáférést.

403 Forbidden: A felhasználó nem jogosult az adott erőforrás lekérésére.

500 Internal Server Error: Hiba történt a szerveren a lekérdezés során, például az adatbázis nem elérhető, vagy valamilyen váratlan belső probléma adódott.

Ez a végpont nemcsak az adatok visszaadásáért felel, hanem a biztonságos működés és a felhasználói élmény szempontjából is kulcsfontosságú, hiszen minden további művelet – például új aktivitás felvétele vagy meglévő módosítása – az itt lekért adatokra épül. A felhasználó szerepkörétől függően a rendszer képes az adatokat testre szabottan visszaadni: például adminisztrátor vagy felhasználó csak a saját aktivitásait láthatja.

Tipikus használati esetek közé tartozik például, hogy a felhasználó minden nap ellenőrzi a tevékenységeit, frissíti a haladást, és ennek alapján a frontend vizuálisan megjeleníti a heti statisztikát vagy a befejezett és folyamatban lévő feladatokat. Ezáltal

a végpont biztosítja az alkalmazás központi működését, és a felhasználó motivációját is segíti a napi célok követésében.

Szokások lekérdezése

A /napicsepp/activities/habits végpont a felhasználó összes szokását lekérdezi. Ez a végpont azokat a szokásokat adja vissza, amelyeket a felhasználó több napon keresztül teljesíthet.

A folyamat a következőképpen zajlik: a felhasználó a belépést követően megkapja a JWT alapú hitelesítő tokent, amelyet minden API kéréshez továbbít. Amikor a kliens lekéri az aktivitáslistát, amiben a szokások szerepelnek, a szerver első lépésként ellenőrzi a tokent, hogy az érvényes-e és a felhasználó jogosult-e a kérés végrehajtására. Ezzel biztosítja, hogy csak az adott felhasználó adataihoz férhessen hozzá más ne tudja lekérni a listát.

A szerver a token érvényességének ellenőrzése után az adatbázisból lekérdezi a felhasználó szokás aktivitásait. Minden aktivitás részletesen tartalmazza a nevét, típusát, nehézségi szintjét, a tevékenység kezdő- és záródátumát, valamint a haladás mértékét. Ezen információk segítségével a felhasználó könnyen áttekintheti, hogy mely tevékenységek vannak folyamatban, melyeket kell frissíteni, illetve hany napot teljesített már belöle.

Az endpoint különböző válaszokkal reagálhat, attól függően, hogy a kérés sikeres volt-e vagy sem:

200 OK: A lekérdezés sikeres volt, és a felhasználó összes aktivitását visszakapja a kliens.

401 Unauthorized: A token érvénytelen, lejárt vagy hiányzik, ezért a rendszer megtagadja a hozzáférést.

403 Forbidden: A felhasználó nem jogosult az adott erőforrás lekérésére.

500 Internal Server Error: Hiba történt a szerveren a lekérdezés során, például az adatbázis nem elérhető, vagy valamilyen váratlan belső probléma adódott.

Ez a végpont nemcsak az adatok visszaadásáért felel, hanem a biztonságos működés és a felhasználói élmény szempontjából is kulcsfontosságú, hiszen minden további művelet – például új aktivitás felvétele vagy meglévő módosítása – az itt lekért

adatokra épül. A felhasználó szerepkörétől függően a rendszer képes az adatokat testre szabottan visszaadni: például adminisztrátor vagy felhasználó csak a saját aktivitásait láthatja.

Tipikus használati esetek közé tartozik például, hogy a felhasználó minden nap ellenőrzi a tevékenységeit, frissíti a haladást, és ennek alapján a frontend vizuálisan megjeleníti a heti statisztikát vagy a befejezett és folyamatban lévő feladatokat. Ezáltal a végpont biztosítja az alkalmazás központi működését, és a felhasználó motivációját is segíti a napi célok követésében.

Új activity létrehozása (POST /napicsepp/activities)

Az /napicsepp/activities POST végpont lehetővé teszi a felhasználók számára, hogy új aktivitásokat rögzítsenek az alkalmazásban. Ez az egyik legfontosabb funkció, hiszen a napi tevékenységek nyomon követése ezen a ponton kezdődik, és az összes további statisztikai és vizuális megjelenítés erre épül.

A kliens elküldi az új aktivitás adatait a szervernek json fomátumban, amely először ellenőrzi a token érvényességét, majd az adatokat validálja. A rendszer a validálás során ellenőrzi, hogy a felhasználó megadott minden szükséges mezőt (például név, típus, nehézségi szint, kezdő- és végdátum). Ezután ellenőrzi, hogy a megadott értékek megfelelnek-e a rendszer szabályainak. Ha minden rendben van, a rendszer az új rekordot az adatbázisba helyezi, és visszajelzést ad a sikeres létrehozásról.

A végpont használata során a felhasználó könnyen hozzáadhatja a napi tevékenységeit, és ezek azonnal megjelennek a frontend felületen, lehetővé téve a haladás nyomon követését és a motiváció fenntartását.

Lehetséges válaszok:

201 Created: Az aktivitás sikeresen létrejött és rögzítésre került.

400 Bad Request: Hibás vagy hiányos adatok miatt nem sikerült létrehozni az aktivitást.

401 Unauthorized: Érvénytelen vagy hiányzó token miatt megtagadva a hozzáférés.

403 Forbidden: A felhasználó nem jogosult új aktivitás létrehozására.

500 Internal Server Error: Szerverhiba miatt nem sikerült az adat mentése.

Tipikus használati eset például, hogy a felhasználó minden reggel felviszi a napi tevékenységeit, amelyek alapján a rendszer vizuálisan frissíti a haladási diagramokat és heti statisztikákat.

Activity módosítása (PUT /napicsepp/activities/:id)

A PUT /napicsepp/activities/:id végpont lehetővé teszi, hogy a felhasználó frissítse egy meglévő aktivitás adatait. Ez fontos funkció, mert gyakran előfordulhat, hogy egy aktivitást hibásan rögzítenek, vagy a felhasználó szeretné módosítani a nehézségi szintet, a kezdő- és végdátumot.

A működés során a kliens elküldi a módosítandó adatokat és az aktivitás azonosítóját a szervernek. A szerver először ellenőrzi a token érvényességét és a felhasználó jogosultságát, majd validálja a beérkező adatokat. Amennyiben minden rendben van, a szerver frissíti az adatbázisban a megfelelő rekordot, és visszajelzést ad a sikeres módosításról.

Ez a végpont kulcsfontosságú a felhasználói kontroll szempontjából, hiszen biztosítja, hogy a felhasználó az adatait mindig pontosan és naprakészen tarthassa. A frontend ezután a módosított adatokat azonnal megjeleníti a felhasználónak, így a vizuális áttekintés és a statisztikák mindig valós állapotot tükröznek.

Lehetséges válaszok:

200 OK: Az aktivitás sikeresen frissítve.

400 Bad Request: Hiányzó vagy hibás adatok miatt a frissítés nem hajtható végre.

401 Unauthorized: Érvénytelen vagy hiányzó token miatt megtagadva a hozzáférés.

403 Forbidden: A felhasználó nem jogosult az adott aktivitás módosítására.

500 Internal Server Error: Szerverhiba a frissítés során.

Tipikus használati eset, amikor a felhasználó korrigálja az aktivitás adatait, például módosítja a befejezés dátumát.

Activity törlése (DELETE /napicsepp/activities/:id)

A DELETE /napicsepp/activities/:id végpont lehetővé teszi, hogy a felhasználó véglegesen eltávolítson egy aktivitást az alkalmazásból. Ez a funkció alapvető a rendszer tisztán tartása és a felhasználói kontroll szempontjából, mivel a felhasználó csak az általa létrehozott tevékenységeket törölheti.

A szerver a token ellenőrzése után meghatározza, hogy a felhasználó jogosult-e a törlés végrehajtására. Amennyiben a jogosultság és a token érvényessége megfellel az elvártnak, a szerver törli az adatbázisból a megadott azonosítóhoz tartozó aktivitást. Ezt követően a felhasználó visszajelzést kap a törlés sikerességéről, és a frontend frissíti az aktivitások listáját, hogy a már törölt elem ne jelenjen meg.

Lehetséges válaszok:

200 OK: Az aktivitás sikeresen törölve.

401 Unauthorized: Hiányzó vagy érvénytelen token miatt a hozzáférés megtagadva.

403 Forbidden: A felhasználó nem jogosult törölni az adott aktivitást.

404 Not Found: A megadott azonosítóhoz nem tartozik aktivitás az adatbázisban.

500 Internal Server Error: Szerverhiba a törlés során.

Tipikus használati eset, amikor a felhasználó eltávolítja a már nem releváns vagy hibásan rögzített aktivitásokat, ezzel biztosítva, hogy az alkalmazásban csak a valós és aktuális adatok maradjanak.

Szerepkörök és jogosultságok

A NapiCsepp alkalmazás három különböző szerepkört különböztet meg: User, Moderator és Admin. A szerepkörök célja, hogy a rendszerben elérhető funkciókhoz eltérő szintű hozzáférést biztosítsanak, ezáltal növelve a biztonságot és a rendszer átláthatóságát. A szerepkörök a felhasználóhoz tartozó JWT tokenben kerülnek eltárolásra, így minden védett végpont esetén a backend a token alapján dönti el, hogy a felhasználó jogosult-e az adott művelet végrehajtására.

A szerepkörök hierarchiája a következőképpen épül fel:

User – alap felhasználó, kizárólag a saját adatait kezelheti.

Moderator – rendelkezik minden User jogosultsággal, továbbá képes más felhasználók adatait megtekinteni és szerkeszteni.

Admin – teljes körű hozzáféréssel rendelkezik, beleértve a rendszerstatisztikák lekérdezését és a felhasználók kezelését.

A szerepkörök célja, hogy a rendszerben végrehajtott műveletek kontrolláltak legyenek, és minden felhasználó csak a számára engedélyezett funkciókat érhesse el.

Funkció / Művelet	User	Moderator	Admin
Saját aktivitások lekérdezése	Igen	Igen	Igen
Saját aktivitások létrehozása	Igen	Igen	Igen
Saját aktivitások módosítása	Igen	Igen	Igen
Saját aktivitások törlése	Igen	Igen	Igen
Más felhasználók aktivitásainak megtekintése	Nem	Igen	Igen
Felhasználók listázása	Nem	Igen	Igen
Felhasználók adatainak szerkesztése	Nem	Igen	Igen
Felhasználók törlése	Nem	Igen	Igen
Rendszerstatisztikák lekérdezése	Nem	Igen	Igen
Admin funkciók elérése	Nem	Igen	Igen
Moderátorok kezelése	Nem	Nem	Igen
Adminok kezelése	Nem	Nem	Igen

Frontend komponensek bemutatása

A NapiCsepp frontend React könyvtár alapú, Vite build eszközzel. A komponensek két fő csoportba sorolhatók: oldalak (pages) és újra felhasználható komponensek (components). Az adatok kezelését a Services és Views réteg végzi.

Sidebar komponens

A Sidebar a navigáció gerince. Elhelyezkedése: src/components/Sidebar/sidebar.jsx. Szerepe: az alkalmazás összes főoldalára mutató link megjelenítése, valamint a bejelentkezett felhasználó szerepkörétől függő menüpontok kezelése.

Kommunikáció

ActivityService.getAllActivities() – az összes aktivitást lekéri.

ActivityService.getAllHabits() – a szokáslistát kéri le.

A lekért adatokból az activityUtils.buildHabitMap() és isAchieved() segítségével számítja ki a függőben lévő feladatok számát.

10 másodpercenként automatikusan frissíti a számlálót, és minden oldalváltáskor is lefut.

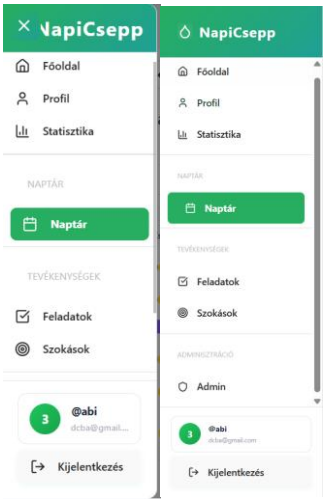
Megjelenített elemek

Navigációs linkek: Főoldal, Profil, Statisztikák, Naptár, Feladatok, Szokások, Admin (csak Admin/Moderator számára).

Függőben lévő feladatok száma badge-ként a Feladatok link mellett.

Kijelentkezés gomb (UserService.logout() hívással).

Hamburger menü kis kijelzőn (reszponzív viselkedés).



3. ábra Sidebar komponens megjelenése mobil(bal) és asztali(jobb) nézetben

AddTaskPopup komponens

Elhelyezkedés: src/components/AddTaskPopup/AddTaskPopup.jsx. Ez az általános célú felugró ablak új feladat létrehozására és meglévő szerkesztésére egyaránt alkalmas (az existingTask prop jelenléte alapján dönti el a módot).

Props

Prop	Leírás
isOpen	Boolean – megnyitja/bezárja a popup-ot.
onClose	Callback – bezáráskor hívódik.
onSuccess	Callback – sikeres mentés után hívódik.
selectedDate	Opcionális – előre kitölti a dátum mezőt.
existingTask	Opcionális – ha meg van adva, szerkesztési módba vált.

Kommunikáció

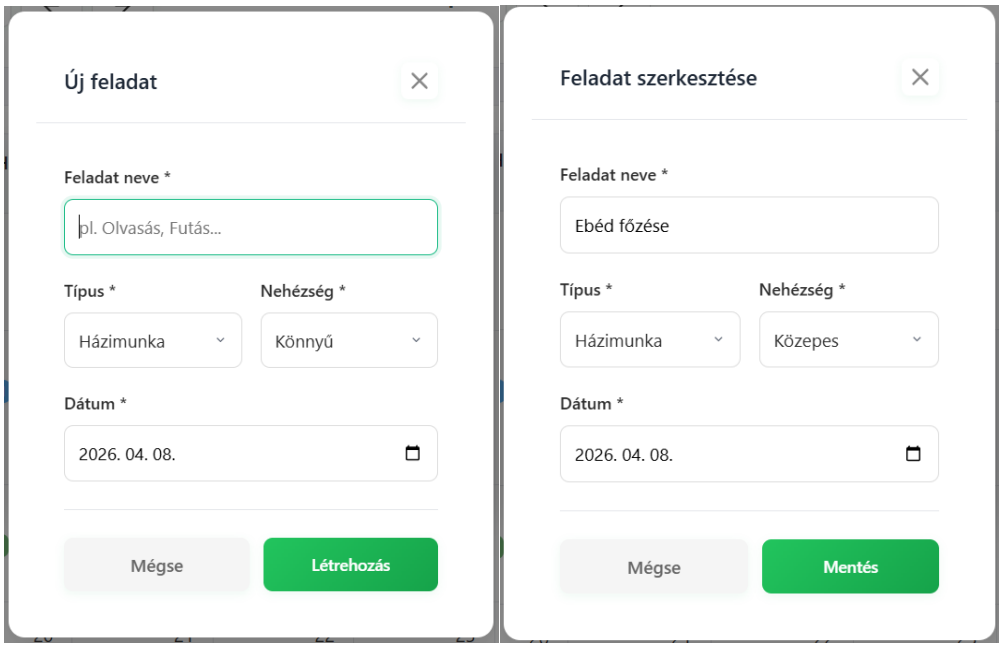
ActivityService.getAllTypes() – típusok lekérdezése (szűri a "Szokás" típust, ha feladatot hozunk létre).

ActivityService.getAllDifficulties() – nehézségi szintek lekérdezése.

ActivityService.createTask(data) – új feladat létrehozása.

ActivityService.updateTask(id, data) – meglévő feladat frissítése.

A showToast() segédfüggvény jelzi a felhasználónak a sikert vagy hibát.



4. ábra Feladatokat kezelő popup komponens létrehozás és szerkesztési módban

EventPopup komponens

Elhelyezkedés: src/components/EventPopup/EventPopup.jsx. Az EventPopup a naptárhoz kapcsolódó esemény létrehozására és szerkesztésére szolgáló felugró ablak. Szorosan együttműködik a naptár nézetek (CalendarManager, DailyView, stb.) komponensekkel.

Kommunikáció

ActivityService.createEvent(data) – új esemény rögzítése.

ActivityService.updateEvent(id, data) – esemény módosítása.

ActivityService.deleteEvent(id) – esemény törlése.

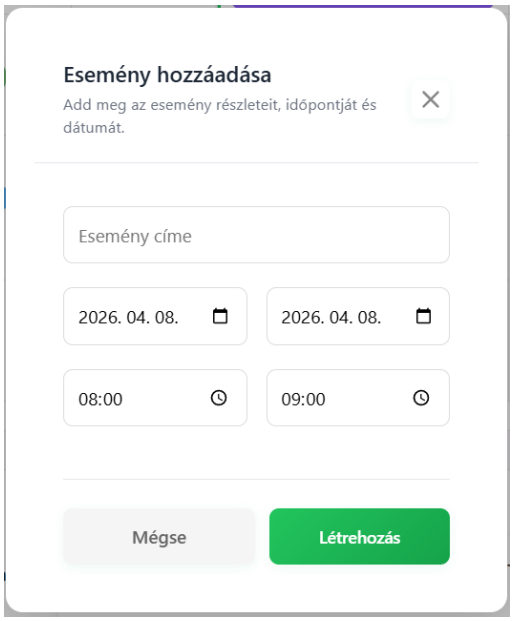
Az EventService osztályon keresztül kezeli az esemény időpontját, színét és leírását.

Megjelenített elemek

Esemény neve, dátuma, időpontja, végső időpontja, kategória színe, leírás mező.

Törlés gomb (csak szerkesztési módban).

Mentés és Mégse gombok.



5. ábra Event popup komponens naptár nézetben

Toast és showToast segédkomponens

Elhelyezkedés: src/components/Toast/Toast.jsx, src/components/Toast/showToast.js. A Toast komponens feladata, hogy a felhasználónak azonnali visszajelzést adjon a műveletek sikerességéről vagy hibájáról. A showToast() egy globális segédfüggvény, amelyet bármely komponensből meghívhatunk.

Használat

showToast("Sikeresen mentve!", "success") – zöld sikeres értesítő.

showToast("Hiba történt!", "error") – piros hibás értesítő.

A Toast automatikusan eltűnik néhány másodperc elteltével.

A komponens nem kommunikál közvetlenül a backenddel; az API-hívó komponensek (AddTaskPopup, EventPopup stb.) hívják meg sikeres vagy sikertelen válasz után.

Manuális tesztek

Backend manuális tesztek

Az alábbi táblázatok a NapiCsepp backend főbb funkcióinak manuális teszteseteit tartalmazzák. Minden teszteset tartalmazza a tesztelendő funkciót, az elvégzett lépéseket, az elvárt eredményt és a tényleges eredményt.

MT-01 – Bejelentkezés érvényes adatokkal

Teszteset azonosító	MT-01
Tesztelt funkció	Bejelentkezés érvényes felhasználói adatokkal
Előfeltétel	Az alkalmazás fut, a felhasználó (abcd@gmail.com) létezik az adatbázisban.
Lépések	1. POST /napicsepp/login endpoint meghívása. 2. Request body: {email: "abcd@gmail.com", password: "1234"} 3. Válasz ellenőrzése: HTTP 200 és token visszaadása.
Tesztadatok	email: abcd@gmail.com, password: "1234"
Elvárt eredmény	HTTP 200 – A szerver visszaadja a felhasználó JWT tokenjét, amelyet a további kérésekhez hitelesítésre kell használni.
Tényleges eredmény	Sikeres – a token megérkezett, a bejelentkezés elvégzett.
Státusz	Megfelelt

MT-02 – Regisztráció hiányzó kötelező mezőkkel

Teszteset azonosító	MT-02
Tesztelt funkció	Regisztráció validáció – kötelező mezők ellenőrzése

Előfeltétel	Az alkalmazás fut, a regisztrációs endpoint elérhető.
Lépések	1. POST /napicsepp/regisztrate endpoint meghívása. 2. Request body: { email: "", password: "", username: "" } 3. Válasz státuszkód és hibaüzenet ellenőrzése.
Tesztadatok	email: (üres) password: (üres) username: (üres)
Elvárt eredmény	HTTP 400 – "Hiányzó kötelező mező(k)" hibaüzenet jelenik meg, a regisztráció nem jön létre.
Tényleges eredmény	Sikeres – a szerver 400-as választ adott vissza a megfelelő hibaüzenettel.
Státusz	Megfelelt

MT-03 – Aktivitás létrehozása érvényes adatokkal

Teszteset azonosító	MT-03
Tesztelt funkció	Aktivitás rögzítése hitelesített felhasználóként
Előfeltétel	A felhasználó bejelentkezett, érvényes JWT token rendelkezésre áll. Az adatbázisban léteznek a megadott típus és nehézségi szint bejegyzések.
Lépések	1. POST /napicsepp/login – token megszerzése. 2. POST /napicsepp/activities meghívása az x-access-tokennel. 3. Request body: {activity_name: "8000 lépés"; activity_type_name: "Mozgás"; activity_difficulty_name: "Közepes"; activity_achive: 0; activity_start_date: "2025-11-11"; activity_end_date: "2025-11-11"; progress_counter: 0} 4. Válasz ellenőrzése.
Tesztadatok	activity_name: "8000 lépés"

	activity_type_name: "Mozgás" activity_difficulty_name: "Közepes" activity_start_date: 2025-11-11 activity_end_date: 2025-11-11
Elvárt eredmény	HTTP 201 – Az aktivitás sikeresen rögzítésre kerül, a válasz tartalmazza az újonnan létrehozott rekord adatait.
Tényleges eredmény	Sikeres – a szerver visszaigazolta a sikeres adatrögzítést.
Státusz	Megfelelt

MT-04 – Felhasználói lista lekérése nem megfelelő jogosultsággal

Teszteset azonosító	MT-04
Tesztelt funkció	Hozzáférés-vezérlés /users endpoint jogosultság ellenőrzés
Előfeltétel	A bejelentkezett felhasználó szerepköre egyszerű felhasználó (nem moderátor, nem admin). Érvényes JWT token rendelkezésre áll.
Lépések	1. POST /napicsepp/login – normál jogú felhasználó tokenjének megszerzése. 2. GET /napicsepp/users meghívása az x-access-tokennel. 3. Válasz státuszkód és hibaüzenet ellenőrzése.
Tesztadatok	email: emptytest@gmail.com (egyszerű felhasználó) password: "1234"
Elvárt eredmény	HTTP 403 – "Csak a moderátor kérheti le!" hibaüzenet jelenik meg. A felhasználói lista nem kerül visszaadásra.
Tényleges eredmény	Sikeres – a szerver 403-as választ adott vissza, az adatok nem kerültek kiszolgálásra.
Státusz	Megfelelt

Frontend manuális tesztek

Az alábbi táblázatok a NapiCsepp frontend főbb funkcióinak manuális teszteseteit tartalmazzák. Minden teszteset tartalmazza a tesztelendő funkciót, az elvégzett lépéseket, az elvárt eredményt és a tényleges eredményt.

MT-01 – Bejelentkezés helyes adatokkal

Mező	Leírás
Teszteset azonosító	MT-01
Tesztelt funkció	Bejelentkezés érvényes adatokkal
Előfeltétel	Az alkalmazás fut, a felhasználó létezik az adatbázisban.
Lépések	1. Navigálás a /login oldalra. 2. E-mail mező kitöltése: felhasznalo@pelda.hu 3. Jelszó mező kitöltése: Titok123 4. Belépés gomb megnyomása.
Elvárt eredmény	Átirányítás a /profile oldalra, JWT token megjelenik a localStorage-ban.
Tényleges eredmény	Sikeres – az oldal átirányít a profilra.
Státusz	Megfelelt

MT-02 – Új feladat hozzáadása

Mező	Leírás
Teszteset azonosító	MT-02
Tesztelt funkció	Új feladat rögzítése az AddTaskPopup felületen
Előfeltétel	A felhasználó bejelentkezett, a /tasks oldal aktív.
Lépések	1. Feladat hozzáadása gomb megnyomása. 2. Feladat neve: "Napi futás".

	3. Típus: Mozgás. 4. Nehézségi szint: Közepes. 5. Dátum kiválasztása: aktuális nap. 6. Mentés gomb megnyomása.
Elvárt eredmény	A feladat megjelenik a listában, sikeres Toast értesítő jelenik meg.
Tényleges eredmény	Sikeres – a feladat azonnal látható a listában.
Státusz	Megfelelt

MT-03 – Feladat törlése

Mező	Leírás
Teszteset azonosító	MT-03
Tesztelt funkció	Meglévő feladat törlése a listából
Előfeltétel	A felhasználó bejelentkezett, legalább egy feladat szerepel a listában.
Lépések	1. Navigálás a /tasks oldalra. 2. Törölni kívánt feladat törlés ikonjára kattintás.
Elvárt eredmény	A feladat eltűnik a listából, a Sidebar számlálója csökken.
Tényleges eredmény	Sikeres – a törölt feladat eltűnik az oldalról.
Státusz	Megfelelt

MT-04 – Szokás létrehozása és haladás követése

Mező	Leírás
Teszteset azonosító	MT-04

Tesztelt funkció	Új szokás rögzítése és a teljesített napok számának ellenőrzése
Előfeltétel	A felhasználó bejelentkezett.
Lépések	1. Navigálás a /habits oldalra. 2. Szokás hozzáadása gomb megnyomása. 3. Szokás neve: "Olvasás 30 percet". 4. Célnapok száma: 30. 5. Kezdő dátum: aktuális nap. 6. Mentés.
Elvárt eredmény	A szokás megjelenik a listában 0/30 teljesített nappal, a haladás sáv 0%.
Tényleges eredmény	Sikeres – a szokás azonnal látható helyes adatokkal.
Státusz	Megfelelt

Automatizált tesztek

Automatizált backend tesztek

A Napicsepp backend automatizált tesztelése Cypress keretrendszerrel valósult meg az e2e végponttesztelés, és jest keretrendszerrel valósult meg az egység tesztek elkészítése. A tesztek két nagy csoportra oszthatók: end-to-end (e2e) tesztek, amelyek a REST API végpontok helyes működését vizsgálják, és komponens tesztek, amelyek az egyes függvények helyes működését vizsgálják.

E2E tesztek

Az e2e tesztek az src/cypress/e2e/ mappában találhatók. Minden tesztelési fájl egy-egy végpontot fed le. A tesztek cy.task(„resetDb”) segédparancsot alkalmaznak, amely az adatbázist egy seed.sql file alapján minden teszt elött újra alapértelmezett állapotra visszaállít.

Tesztfile	Tesztek száma	Lefedett végpont
activity_tests.cy.ts	21	/napicsepp/activities/… Lefedi az összes activity végpontot, ami elvégzi a CRUD műveleteket
difficulty_test.cy.ts	1	/napicsepp/difficulties Lefedi a GET kérést, ami visszaadja a nehézségi szinteket
event_tests.cy.ts	12	/napicsepp/events/… Lefedi az eventekhez tartozó CRUD műveleteket végző endpointokat
overview_tests.cy.ts	4	/napicsepp/overview Lefedi a GET kérést, ami visszaadja az áttekintést
statistic_tests.cy.ts	10	/napicsepp/statistic/… Lefedi a statisztikákkal való vissaztérésért felelős endpointokat
type_tests.cy.ts	1	/napicsepp/types Lefedi a tipusok visszatérésére szolgáló endpointot
user_tests.cy.ts	16	/napicsepp/users/… Lefedi a felhasználók CRUD műveleteihez és a loginhoz való endpointokat

Komponens tesztek

A komponens tesztek az backend/tests mappában találhatók. Ezek a tesztek mokkolt adatbázissal dolgoznak és követik a 3A technikát. (Arrange, Act, Assert). A tesztek csoportokba vannak bontva ugyanugy mint a backend Repository, Service, Controller.

A tesztek 100%-os lefedettségi szinttel rendelkeznek, amit a jest beépített code coverage szolgáltatása mutatott meg.

Repository tesztek:

Teszt file	Lefedett metodusok
activity_repository.spec.ts	Lefedi az összes fontos adatbázis-műveletét (lekérdezés, beszúrás, törlés, frissítés, valamint hibakezelés és tranzakciók működése) ellenőrzi mockolt MySQL kapcsolaton keresztül.
difficulty_repository.spec.ts	Lefedi a getDifficulties metódusának működését ellenőrzi, beleértve a sikeres adatlekérdezést, az üres eredményt és a hibakezelést mockolt MySQL kapcsolaton keresztül.
event_repository.spec.ts	Lefedi az összes fő adatbázis-műveletét (lekérdezés, beszúrás, törlés, frissítés, valamint tranzakciókezelés és hibakezelés) ellenőrzi mockolt MySQL kapcsolaton keresztül.
overview_repository.spec.ts	Lefedi a getOverview metódusának működését ellenőrzi, beleértve a sikeres adatlekérdezést, az üres eredményt és a hibakezelést mockolt MySQL kapcsolaton keresztül.
statistic_repository.spec.ts	Lefedi a rendszer- és profilstatisztikai lekérdezéseinek működését ellenőrzi, beleértve a sikeres adatlekérdezést, az üres eredményt és a hibakezelést mockolt MySQL kapcsolaton keresztül.
type_repository.spec.ts	Lefedi a getTypes metódusának működését ellenőrzi, beleértve a sikeres adatlekérdezést, az üres eredményt és a hibakezelést mockolt MySQL kapcsolaton keresztül.
user_repository.spec.ts	Lefedi az összes fontos felhasználókezelési műveletét (bejelentkezés, lekérdezések, létrehozás,

	törlés, frissítés, jogosultságkezelés és hibakezelés) ellenőrzi mockolt MySQL kapcsolaton keresztül.

Service tesztek:

Teszt file	Lefedett metodusok
activity_service.spec.ts	Ez a teszt lefedi az ActivityService teljes üzleti logikáját, beleértve az adatlekérések, létrehozás, törlés és frissítés működését, valamint az érvényesítést és hibakezelést mockolt repository használatával.
difficulty_service.spec.ts	Ez a teszt lefedi a DifficultyService üzleti logikáját, beleértve a nehézségek lekérését, az üres vagy null eredmények kezelését, valamint a hibák propagálását mockolt repository használatával.
event_service.spec.ts	Ez a teszt lefedi az EventService üzleti logikáját, beleértve az események lekérését, létrehozását, frissítését és törlését, valamint a bemeneti adatok érvényesítését és a hibakezelést, mockolt repository használatával.
overview_service.spec.ts	Ez a teszt lefedi az OverviewService üzleti logikáját, beleértve a felhasználói összefoglalók lekérését, a visszaadott adatok kezelését és az esetleges hibák propagálását mockolt repository használatával.
statistic_service.spec.ts	Ez a teszt lefedi a StatisticService üzleti logikáját, beleértve a rendszer- és profilstatisztikák lekérését, a moderátori jogosultságok ellenőrzését, az üres vagy null eredmények kezelését, valamint a hibakezelést mockolt repository-k használatával.
type_service.spec.ts	Ez a teszt lefedi a TypeService üzleti logikáját, beleértve a típusok lekérését, az üres vagy null eredmények kezelését, valamint a hibakezelést mockolt repository használatával.

user_service.spec.ts	Ez a teszt lefedi a UserService üzleti logikáját, beleértve a felhasználók bejelentkezését, regisztrációját, adatlekérést, szerkesztést, törlést, valamint a jogosultság-ellenőrzést, érvényesítést és hibakezelést mockolt repository és jwt használatával.

Controller tesztek:

Teszt file	Lefedett metodusok
activity_controller.spec.ts	Ez a teszt lefedi az ActivityController összes végpontját, beleértve a lekéréseket, létrehozást, módosítást és törlést, valamint az összes lehetséges hibakezelési esetet mockolt ActivityService használatával.
difficulty_controller.spec.ts	A teszt ellenőrzi, hogy a DifficultyController megfelelően kezeli a sikeres lekérést és minden lehetséges hibás esetet mockolt DifficultyService használatával.
event_controller.spec.ts	Ez a fájl az EventController összes CRUD-műveletét teszteli, lefedve a sikeres útvonalakat és a különböző hibakezelési eseteket.
overview_controller.spec.ts	Ez a fájl az OverviewController getOverview metódusát teszteli, lefedve a sikeres adatlekérést, az üres eredményt, valamint különböző hibakezelési eseteket.
statistic_controller.spec.ts	Ez a fájl a StatisticController összes metódusát teszteli, lefedve a rendszer-, profil- és felhasználói statisztikák lekérését, valamint a különböző hibakezelési eseteket.
type_controller.spec.ts	Ez a fájl a TypeController getTypes metódusának működését teszteli, ellenőrizve a sikeres adatlekérést és a különböző hibakezelési eseteket.

user_controller.spec.ts	Ez a fájl a UserController különböző metódusainak működését teszteli, beleértve a bejelentkezéstlogin), regisztrációt (register), felhasználói adatok lekérését (getUser, getAllUser), szerkesztést (putUser), moderátorok lekérését (getModerators) és törlést (deletUser), valamint minden lehetséges sikeres és hibás állapot kezelését.

Automatizált frontend tesztek

A NapiCsepp frontend automatizált tesztelése Cypress keretrendszerrel valósult meg. A tesztek két nagy csoportra oszthatók: end-to-end (e2e) tesztek, amelyek a teljes alkalmazás folyamatait vizsgálják, és komponens tesztek, amelyek az egyes React komponenseket izoláltan(elszigetelten) tesztelik.

E2E tesztek

Az e2e tesztek az src/cypress/e2e/ mappában találhatók. Minden tesztelési fájl egy-egy oldalt vagy főbb funkcionalitást fed le. A tesztek cy.visitProtected() segédparancsot alkalmaznak, amely bejelentkezési lépés után navigál a védett oldalakra.

Tesztfájl	Tesztek száma	Lefedett funkcionalitás
login.cy.js	30	Bejelentkezési oldal megjelenése, mezők kitölthetősége, hibaüzenetek, sikeres login átirányítás.
activities.cy.js	55	Feladatok oldal betöltése, CRUD műveletek, navigáció.
calendar.cy.js	60	Naptár különböző nézetek (nap, hét, hónap), esemény kezelés.
homepage.cy.js	25	Főoldal betöltése, napi összefoglaló megjelenítése.

statistics.cy.js	35	Statisztika oldal diagramjai, adatok megjelenítése.
admin.cy.js	65	Admin panel: felhasználók listázása, szerkesztése, törlése.
profile.cy.js	25	Profiloldal adatainak megjelenítése, szerkeszthetőség.
daily_events.cy.js	27	Napi nézet esemény blokkjainak, mini popupjának és közelgő események panel-jének tesztelése.
homepage_events.cy.js	24	Főoldal mai és közelgő események szekciójának, mini popupjának és tör-lés/szerkesztés funkcióinak tesztelése.
sidebar.cy.js	25	Sidebar navigációs linkek, védett út-vonalak és layout struktúra tesztelése.

Komponens tesztek

A komponens tesztek az src/cypress/component/ mappában találhatók. Ezek a tesztek a cy.mount() API segítségével renderlik az egyes React komponenseket, és el-lenőrzik azok megjelenését és viselkedését izoláltan, háttér API-hívás nélkül.

Tesztfájl	Lefedett komponens / funkció
components.cy.jsx	PasswordInput (jelszó toggle), DataItem (label+ér-ték), StatItem (statisztika megjelenítő).
login.cy.jsx	LoginView bejelentkezési form renderelése.
register.cy.jsx	RegisterView regisztrációs form renderelése.
homepage.cy.jsx	Főoldal komponens megjelenése.
statistics_class.cy.jsx	Statisztika osztály logikájának tesztelése.
show_toast.cy.jsx	Toast értesítők megjelenése sikeres és hibás esetben.

calendar_manager.cy.jsx	CalendarManager osztály metódusainak tesztelése.

A komponens tesztek egyik kiemelkedő példája a PasswordInput tesztelése: ellenőrzik, hogy az input alapértelmezetten password típusú, a toggle gombra kattintva text típusra vált, majd dupla kattintásra visszavált. Ez a viselkedés biztosítja a felhasználóbarát jelszóbeviteli élményt a bejelentkezési és regisztrációs felületeken.

Felhasználói dokumentáció

Szerepkörök bemutatása

Felhasználó (User)

Az alapszintű felhasználó regisztráció után azonnal hozzáférhet az alkalmazás fő funkcióihoz. Létrehozhat feladatokat és szokásokat, megtekintheti és szerkesztheti saját adatait, kezelheti naptárát és eseményeit, valamint statisztikákat tekinthet meg saját aktivitásairól.

Moderátor (Moderator)

A moderátor rendelkezik az alap felhasználói jogosultságokkal, ezen felül láthatja és szerkesztheti más felhasználók adatait is. Az Admin panelen keresztül elérheti a felhasználók listáját, és szükség esetén módosíthatja azok adatait.

Adminisztrátor (Admin)

Az adminisztrátor teljes körű hozzáféréssel rendelkezik. A moderátori jogosultságokon túl képes moderátorokat kezelni, valamint bárkit kinevezhet adminná és törölhet is adminokat.

Esetek bemutatása

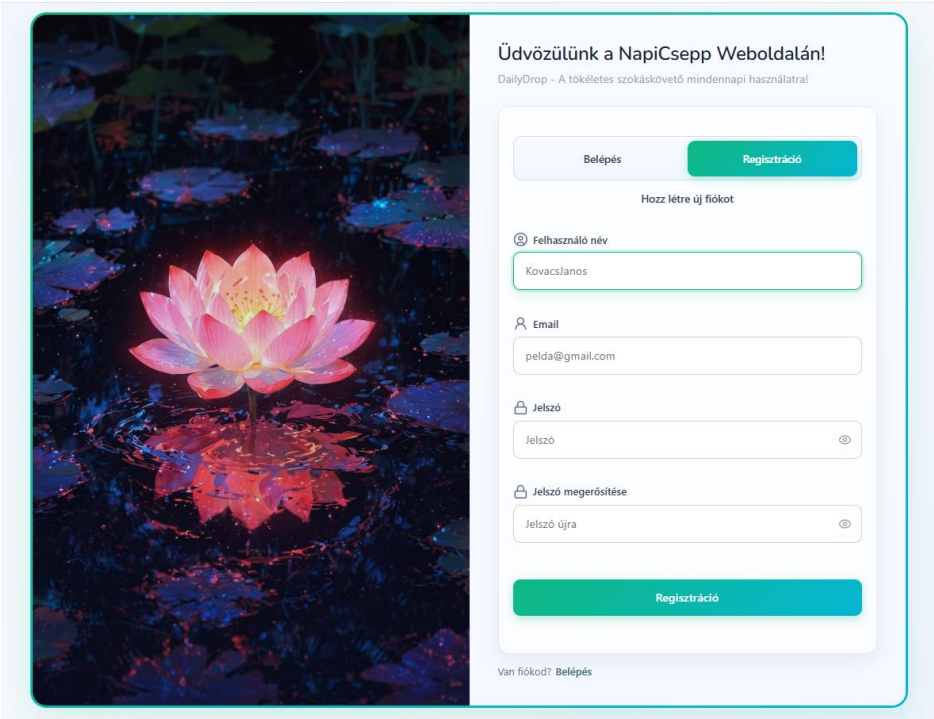
Regisztráció

Az alkalmazás eléréséhez először regisztrálni kell. A regisztrációs oldalon (/register) a felhasználó megadja a szükséges adatokat.

A regisztráció lépései:

1. Navigálj a /register oldalra (vagy kattints a 'Még nincs fiókod? Regisztráció' linkre).
2. Töltsd ki a megjelenő regisztrációs űrlapot:
    - Felhasználónév: egyedi név, szóköz nélkül (pl. KovacsJanos).
    - Email cím: érvényes email formátum (pl. pelda@gmail.com). Ha az email cím már foglalt, hibaüzenet jelenik meg.
    - Jelszó: a szem ikonra kattintva megjelenítheted a begépelt karaktereket.
    - Jelszó megerősítése: pontosan ugyanazt a jelszót kell megadni. Ha a kettő nem egyezik, hibaüzenet jelenik meg.

Kattints a Regisztráció gombra. Sikeres regisztráció esetén a rendszer automatikusan bejelentkeztet és a Főoldalra irányít.



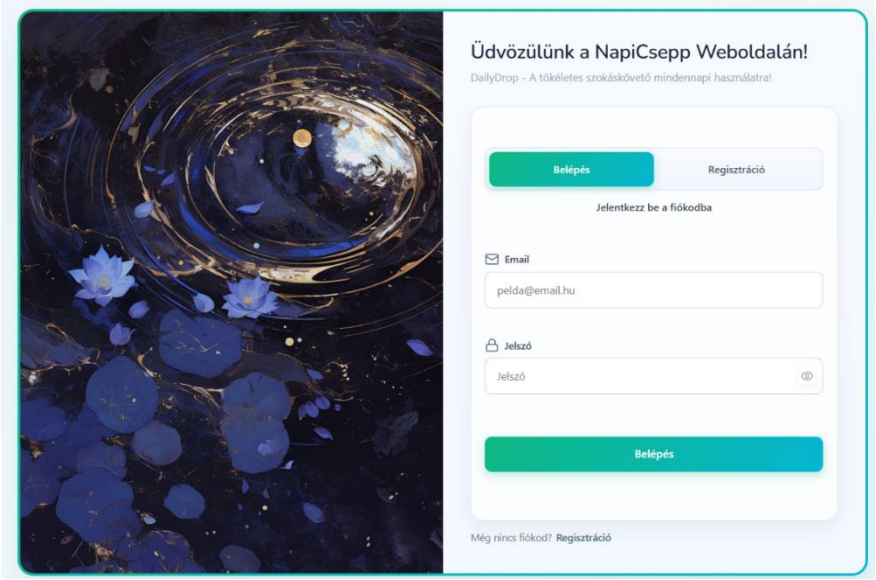
6. ábra – Regisztrációs oldal megjelenése

Bejelentkezés

A visszatérő felhasználók a /login oldalon adják meg e-mail címüket és jelszavukat. Sikeres belépés után a rendszer JWT tokent állít ki, amelyet a böngésző localStorage-ban tárol, és a Főoldalra irányítja a felhasználót.

A bejelentkezés lépései:

3. Navigálj a /login oldalra.
4. Add meg az email-cím és jelszó mezőket. A szem ikonra kattintva megjelenítheted a begépelt jelszót.
5. Kattints a zöld Belépés gombra.
6. Sikeres bejelentkezés esetén a rendszer a Főoldalra (/) navigál, az oldalsáv megjelenik a menüpontokkal.



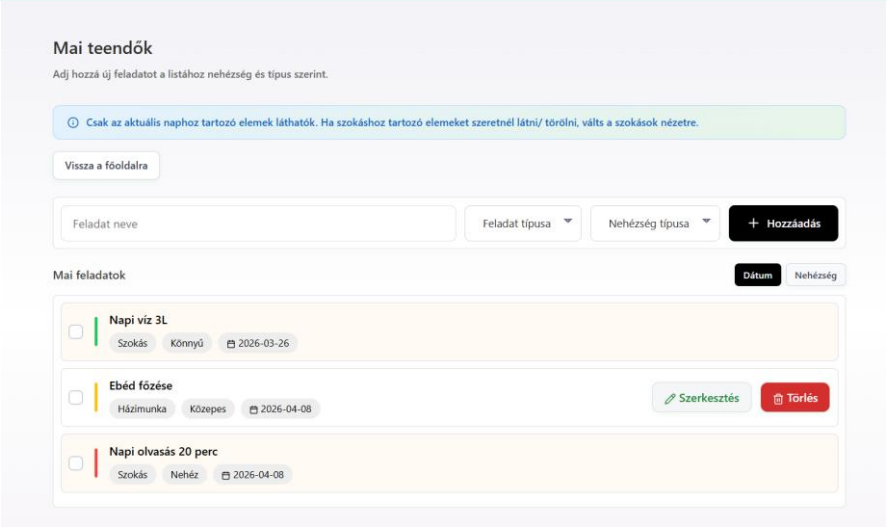
7. ábra – Bejelentkezési oldal

Napi feladatok kezelése

A /tasks oldalon a felhasználó láthatja az összes aktív feladatát. Az "Új feladat" gombra kattintva megnyílik az AddTaskPopup, ahol megadható a feladat neve, típusa, nehézségi szintje és dátuma. A lista minden feladatnál mutatja annak típusát, nehézségi szintjét és állapotát (elvégzett/folyamatban). A pipa gombbal a feladat elvégzettnek jelölhető.

Automatikus frissítés

A haladásszámláló minden éjfélkor automatikusan frissül nem kell kézzel léptetni. Ha egy szokás időtartama lejárt, átkerül a Lejárt szokások szekcióba, ahol lehet módosítani hogy jó legyen a dátum vagy végleg törölni.



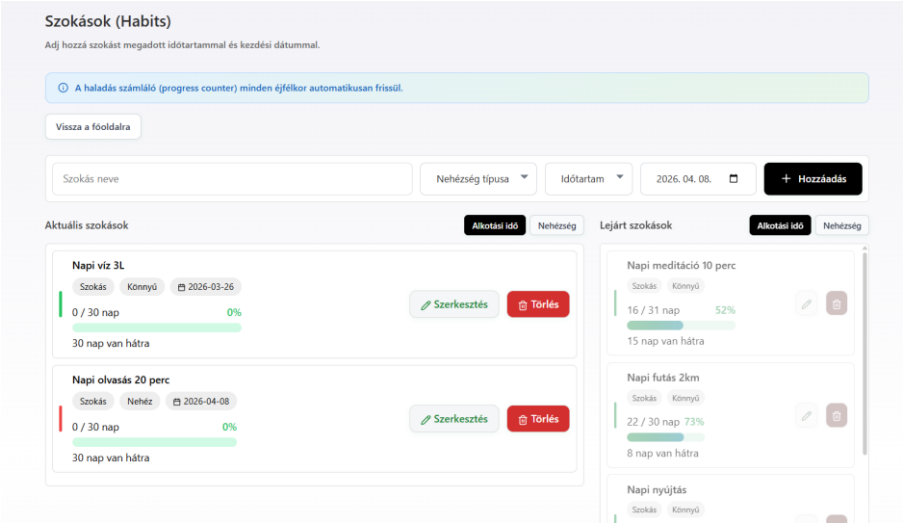
8. ábra – Feladatok oldal

Szokások követése

A /habits oldalon hosszú távú szokások rögzíthetők meghatározott célnapszámmal (pl. 30 napos olvasási szokás). Az oldal minden szokásnál mutatja a haladást (pl. 12/30 teljesített nap), és vizuálisan jelzi, ha egy szokás a célnál tart. Ha az összes célnap teljesített, a szokás automatikusan átkerül az elvégzett kategóriába, és eltűnik az aktív listából.

Automatikus frissítés

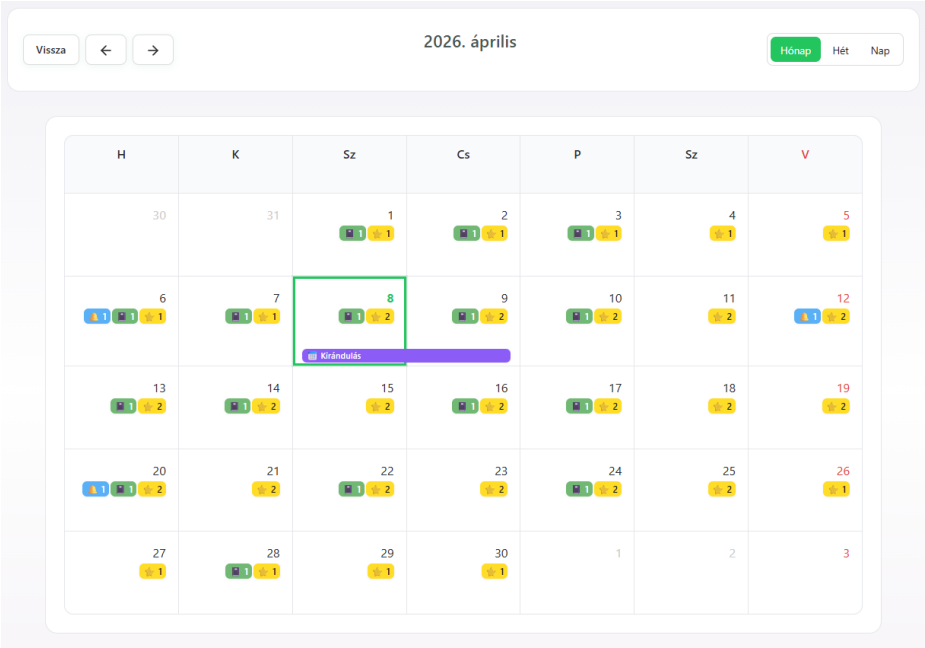
A haladásszámláló minden éjfélkor automatikusan frissül nem kell kézzel léptetni. Ha egy szokás időtartama lejárt, átkerül a Lejárt szokások szekcióba, ahol csak törölni lehet.



9. ábra – Szokások oldal haladás jelzővel

Naptár és eseménykezelés

A /calendar oldalon a felhasználó négy nézetben tekintheti meg eseményeit: nap, hét, hónap és kombinált nézet. Eseményt az EventPopup segítségével hozhat létre: megadja a nevet, időpontot, leírást és kategória színt. Az esemény azonnal megjelenik a naptárban, és a felhasználó bármikor szerkesztheti vagy törölheti azt.



10. ábra - Naptár havi nézet eseményekkel

Statisztikák

A Statisztikák oldalon (/statistics) részletes betekintést nyersz a teljesítményedbe napi és heti szinten:

- 4 összesítő kártya: összes feladat, befejezett, összes esemény, aktív szokások.
- Befejezési arány: progress-sáv mutatja a teljesített feladatok százalékát, alatta befejezett és függőben lévő számok.
- Nehézségi eloszlás: Magas, Közepes, Alacsony nehézségű feladatok száma külön sorokban.
- Heti összesítő: heti összes feladat, befejezett, függőben lévő, napi átlag.
- Heti áttekintés (diagram): oszlopdiagram hétfőtől vasárnapig, zöld = elvégzett, kék = függőben.

Profil kezelése

A Profil oldalon (/profile) megtekintheted a fiókadataidat: felhasználónév, email-cím (csillagokkal), fiók típus (szerepkör), regisztráció dátuma, és 4 profil statisztikai kártya (összes feladat, befejezett aktivitások, aktív szokások, havi események).

Adatok szerkesztése: kattints az Adatok szerkesztése gombra majd, popup formában módosítható a felhasználónév és email-cím, ezután kattints a Mentés gombra.

Admin panel részletes kezelési útmutató

Az admin panel (/admin) csak admin és moderátor jogosultsággal érhető el. A panel a felhasználók kezeléséhez és statisztikáinak megtekintéséhez nyújt eszközöket.

Felhasználók listája és keresés:

Az oldal tetején négy összesítő kártya jelenik meg: összes felhasználó, adminok száma, moderátorok száma, felhasználók száma. A táblázat felett kereső mező és szerepkör-szűrő áll rendelkezésre.

- Keresés: gépeld be a felhasználó nevének vagy email-címének bármely részletét a táblázat valós időben szűr.

- Szerepkör szűrő: az 'Összes szerepkör' legördülőből választhatsz admin / moderator / user szűrést.
- Lapozás: a táblázat alján állítható a sorok száma oldalanként, és a lapszámozó gombok navigálnak.

Felhasználó szerkesztése:

1. Kattints a ceruza ikonra a szerkeszteni kívánt sor végén.
2. A popup közepén megjelenik a szerkesztő form a jelenlegi adatokkal kitöltve.
3. Módosítsd a felhasználónevet, email-címet vagy szerepkört (User / Moderátor / Admin).
4. Kattints a Mentés gombra PUT kérés indul, a táblázat frissül.
5. A popup bezárható a Mégse gombbal, az X gombbal, vagy a háttérre kattintva.

Felhasználó törlése

1. Kattints a kuka ikonra a törölni kívánt sor végén.
2. Megerősítő popup jelenik meg: 'Biztosan törölni szeretnéd?' és 'Ez a művelet nem vonható vissza.'
3. Kattints a Törlés megerősítése gombra -- DELETE kérés indul.
4. A rendszer eltávolítja a felhasználót az összes adatával együtt.

Fontos

Az admin saját fiókját nem törölheti. Moderátor nem törölheti az admint a kuka ikon számára nem látható.

Felhasználói statisztika megtekintése:

Kattints egy felhasználó sorára az /admin/user/:id oldalra navigál, ahol megtekinthető:

- Profil kártya: felhasználónév, email, szerepkör (badge), csatlakozás dátuma.

- Összesítő számlálók: összes tevékenység, teljesített, napi feladatok, havi események.
- Teljesítési arányok: napi és heti progress-sáv százalékkal.
- Nehézségi eloszlás: Könnyű, Közepes, Nehéz feladatok száma és arányuk.
- Heti összesítés: heti feladatok, heti teljesített, heti arány százalékban.

Összefoglalás

A NapiCsepp projekt eredménye egy teljes értékű, szokáskövető és produktivitáskezelő webalkalmazás, amely modern technológiákat alkalmaz mind a frontend, mind a backend oldalon.

A rendszer négy fő funkcionális pillérre épül: a napi feladatkezelésre (types, difficulties, CRUD), a szokáskövetésre (időtartamalapú progress, automatikus éjféli frissítés cron job-bal), az eseménynaptárra (háromféle nézet, CRUD) és a statisztikákra (napi/heti összesítők, nehézségi eloszlás, diagram). A három szerepkör (User, Moderátor, Admin) lehetővé teszi a platform biztonságos üzemeltetését.

A backend Node.js + TypeScript + Express.js kombinációra épül, háromrétegű (Repository-Service-Controller) architektúrával, JWT-alapú hitelesítéssel. A frontend React 19 alapú SPA, Vite build eszközzel, amely a Services réteggel kommunikál a backend API-val.

A tesztelés széleskörű: a backend 100%-os Jest kódlefedettséggel és Cypress E2E tesztekkel rendelkezik és a backend manuális teszteset egy Excel táblában dokumentált; a frontend Cypress E2E és komponenstesztek fedik le. A frontend manuális teszteset egy Excel táblában dokumentált.

A Docker Compose alapú telepítés egyetlen paranccsal indítható, ami jelentősen egyszerűsíti a fejlesztést, a tesztelést és az üzembe helyezést.

A projekt egyik legnagyobb kihívása a kommunikáció és a verziókezelés koordinációja, különösen a téli szünetben, amikor a csapat tagjai két különböző helyszínen dolgoznak. A GitHub és a rendszeres egyeztetések tartják fenn a fejlesztés folyamatosságát

Kitekintés

A NapiCsepp jelenlegi verziója egy stabil, teljes körű MVP (Minimum Viable Product), amely az összes tervezett alapfunkciót megbízhatóan megvalósítja. A rendszer architektúrája azonban tudatosan úgy épül fel, hogy támogassa a távlati fejlesztési irányokat és a funkcionális bővítést.

A felhasználói élmény fokozása érdekében a jövőben a naptárkezelés túllép az egyszeri bejegyzéseken, és támogatja a heti vagy havi rendszerességgel ismétlődő eseményeket. A platform hatékonyságát egy push- és email-alapú értesítési rendszer növeli, amely időben figyelmezteti a klienseket a közelgő határidőkre és az elmaradt rutinfeladatokra. A motiváció fenntartását egy „szokás-streak" (sorozat-számláló) mechanizmus szolgálja, amely vizuálisan is rögzíti, hány egymást követő napon sikerült az adott tevékenységet teljesíteni. Valamint a más felhasználók által kreált szokások is megjelenhetnek, hogy ihletet adjon a felhasználónak. Azzal is a felhasználó élmény bővíthető hogyha majd a felhasználó fogja tudni a típusukat meghatározni. Ezt egészítik ki a hosszú távú, havi és éves statisztikai összesítők, amelyek a részletes adatmegjelenítésen keresztül segítik a fejlődés nyomon követését.

A produktivitást a feladatcímkék és a fejlett szűrési feltételek bevezetése teszi gördülékenyebbé, lehetővé téve a prioritások (pl. sürgős, fontos) és kategóriák szerinti rendszerezést. A kényelmi funkciók sorát a profil oldalon választható rendszerszintű sötét mód bővíti.

A későbbiekben egy mobil applikáció készítése lenne a cél tekintve hogy ez az az applikáció amihez az ember minden nap akár többször is hozzáférne.

Felhasznált források és szoftver bibliográfia

Technológia / Forrás	URL / Hivatkozás
Node.js Documentation	https://nodejs.org/en/docs
Express.js Documentation	https://expressjs.com
TypeScript Handbook	https://www.typescriptlang.org/docs

Technológia / Forrás	URL / Hivatkozás
React Documentation	https://react.dev
React Router DOM	https://reactrouter.com
Vite Documentation	https://vitejs.dev
MySQL 8.0 Reference Manual	https://dev.mysql.com/doc/refman/8.0/en
Cypress Testing Framework	https://docs.cypress.io
Jest Testing Framework	https://jestjs.io/docs/getting-started
JSON Web Tokens (JWT)	https://jwt.io/introduction
jsonwebtoken npm package	https://github.com/auth0/node-jsonwebtoken
mysql2 npm package	https://sidorares.github.io/node-mysql2/docs
bcrypt npm package	https://www.npmjs.com/package/bcrypt
node-cron npm package	https://github.com/node-cron/node-cron
Docker & Docker Compose Documentation	https://docs.docker.com
MDN Web Docs -- HTML & CSS referencia	https://developer.mozilla.org
Phillippa Lally et al. "How habits are formed"	European Journal of Social Psychology, 2009 - a 66 napos szokáskutatás

Mellékletek

Az alábbi dokumentumok és forrásfájlok a GitHub repository-ban találhatók és hivatkozással elérhetők. Az alap mappa a /Napicsepp/Tervezés. Ahol 2 fő mappa található backend és frontend, ami meg simán a tervezés mappában van az mindkettőhöz tartozik mint például ez a dokumentáció.

Leírás	Elérési út
Frontend manuális tesztek	Tervezés/Frontend/NapiCsepp_Frontend_Manualis_Tesztek.xlsx
Frontend cypress e2e tesztek	Projekt/Frontend/cypress/e2e/…cy.js
Frontend cypress unit tesztek	Projekt/Frontend/cypress/component/…cy.jsx
Frontend látványtervek (asztalra, mobilra)	Tervezés/Frontend/Látványtervek
Frontend REST API doksi	Tervezés/Frontend/Napicsepp_Frontend_Dokumentáció.docx
UseCase diagramm	Tervezés/Frontend/Usecase_diagram.png
Sequence diagramok	Tervezés/Sequence
Backend manuális tesztek	Tervezés/Backend/mmanualis_teszt_jegyzokonyv.xlsx
Backend cypress e2e tesztek	Projekt/Backend/cypress/e2e/…cy.ts
Backend jest unit tesztek	Projekt/Backend/tests

Leírás	Elérési út
Adatbázis tervek ER diagram, Relációs diagram és példa adatok excelben	Tervezés/Backend/Adatbázis_terv_projekt
Backend osztály diagram	Tervezés/Backend/Backend_uml.png
Backend osztálydiagramok elemenként	Tervezés/Backend/Backend class diagrams
Backend automatizált unit tesztek részletes dokumentáció	Tervezés/Backend/Automatizált unit tesztek
Backend automatizált e2e tesztek részletes dokumentáció	Tervezés/Backend/Automatizált végpont tesztek
Rest Api dokumentáció	Tervezés/Backend/Napicsepp_Endpoints
Részletes backend osztály dokumentáció	Tervezés/Backend/Részletes backend osztály dokumentáció
Specifikáció	Tervezés/Specifikáció végleges